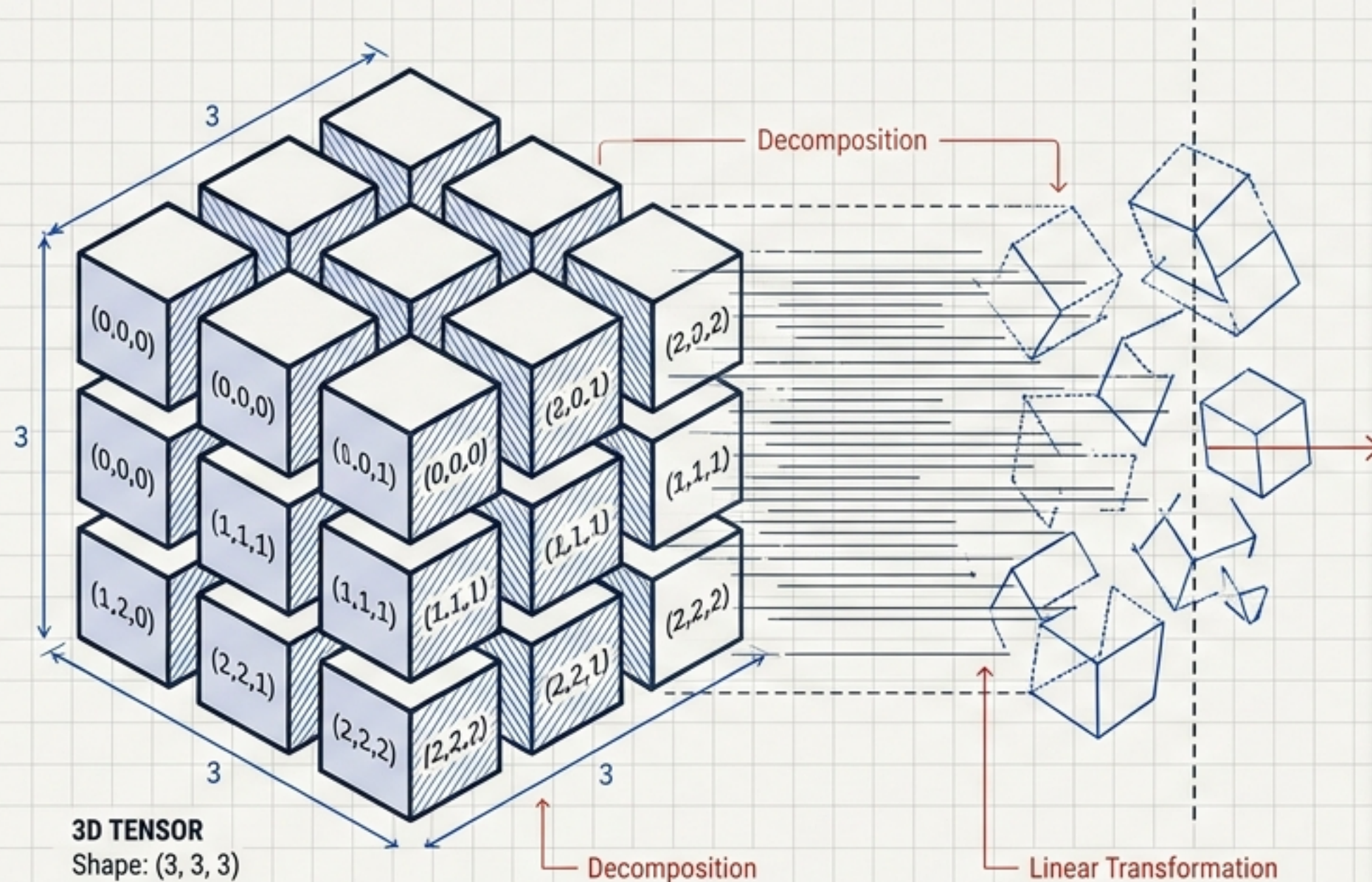


Building the intuition behind every AI algorithm through code.

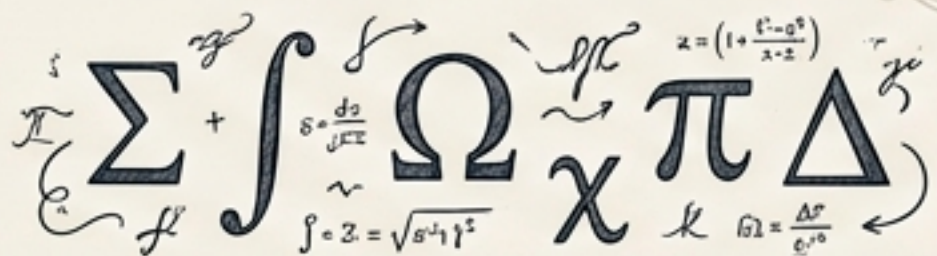
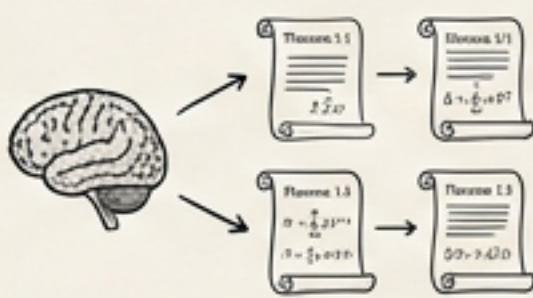
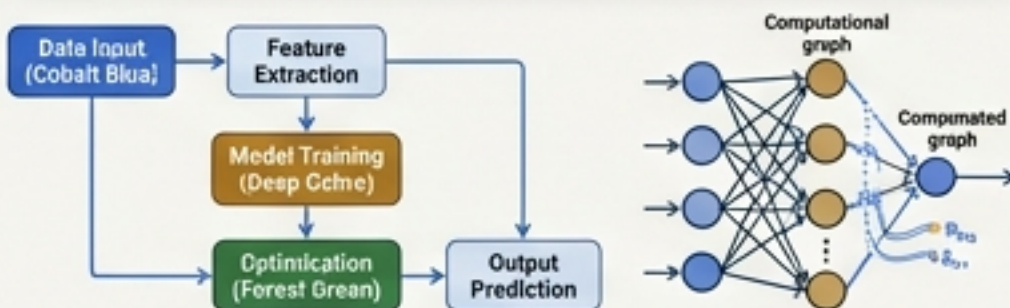
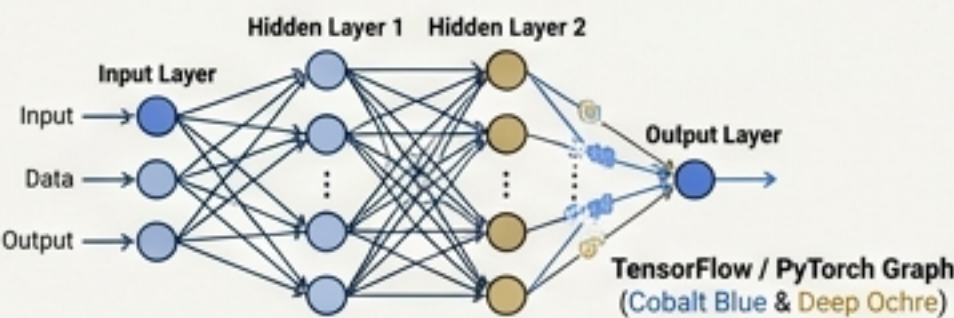
A blueprint for understanding deep learning.

Phase 1: Math Foundations
The intuition behind every AI algorithm, through code — not textbooks.



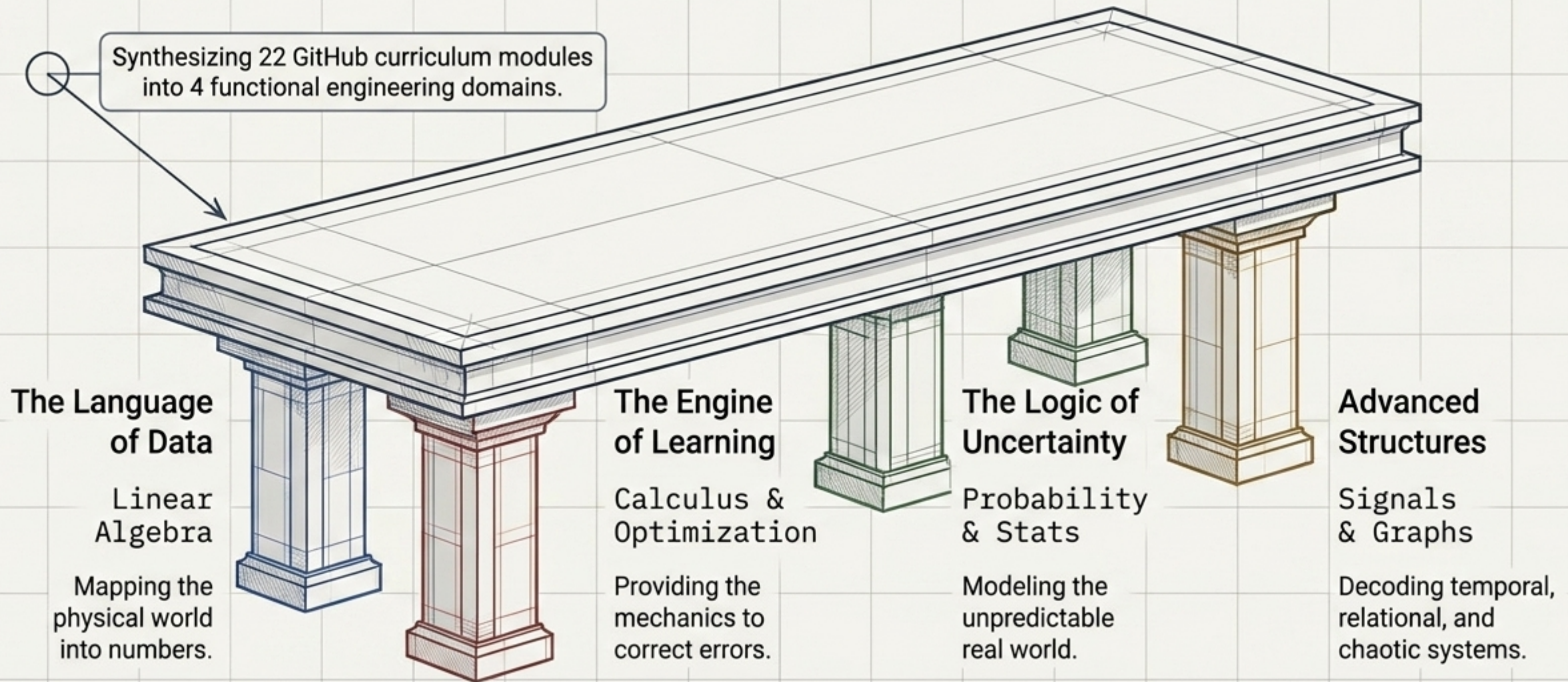
```
1 # Phase 1: Math Foundations
2 # Tensor Decomposition & Reassembly
3
4 import torch
5
6 def create_and_transform_tensor():
7     # Initialize 30 tensor (Data)
8     data_tensor = torch.arange(27).reshape(3, 3, 3)
9
10    # Simulate fracture and linear transformation (Architecture)
11    # Flattening the tensor
12    flat_vector = data_tensor.view(-1)
13
14    # Reassembly into structured code representation
15    # Example operation: Matrix Multiplication (Calculus/Linear Algebra)
16    weights = torch.randn(27, 10) # Architectural elements
17    output = torch.matmul(flat_vector.float(), weights)
18
19    return output
20
21
22 # Execute the transformation
23 result = create_and_transform_tensor()
24 print("Structured Output Shape:", result.shape)
```

We trade manual calculation for algorithmic intuition.

The Academic Textbook		The AI Engineering Approach	
Abstract Greek Symbols		Executable Python Code	<pre>1 import numpy as np 2 3 def compute_loss(y, y_pred): 4 return np.mean((y - y_pred)**2) → Loss: 0.0 5 6 x = np.array([1, 2, 3]) 7 y = np.array([1, 4, 9]) 8 y_pred = compute_loss(y, y_pred) → Loss: 0.0</pre>
Formal Proofs & Memorization	Theorem 1.1: Let X be a set. If $1 \leq X = X$, then a computational notation is a formal proof. $\{s, E\}$. Then $\sum_{i=1}^n f(s-i) \text{ and } f_y = s \leq X.$ Then $f, s \leq 1$ be a theorem as A arises $X \geq z(u)$, and use as a test as approximating problem in k in $\mathcal{R}_{1,1}$, and $\in X$. Q.E.D. 	Algorithmic Intuition & Execution	
Manual Derivations	$\begin{array}{r} 135-5 \\ 21 \overline{) 556-2} \\ \underline{-150} \\ 145 \\ \underline{-248} \\ 339 \\ \underline{219} \\ 0 \end{array}$ $\int f(x) dx = \int f(x=x) dx$ $= \frac{d}{dx} = 3 \int \frac{f(x=x) dx}{dx}$ $= \int f(x=x(x)) = \frac{1}{2} \frac{f(x-1x-1)}{x^5} dx$ $\frac{x^2}{x^3} = \frac{1^5}{x^{2+3}} + \frac{x^2}{x^{2+3}} = \frac{1}{x^5} + \frac{x^2}{x^5}$ $x^2 = -x((z-2r-3)^5 + n)$ $= (x-3x-5) - 4^5 - 32$ $= -65 - x^4 + 15$ $= -4 + 12 + 12$ $= -2x + 25$	Automated Computational Graphs	
Academic Theory		Deployable AI Models	

Math is the blueprint. Code is the building. **We start building on day one.**

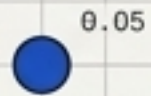
Twenty-two disparate mathematical concepts form a four-pillar architectural system.



Linear algebra acts as the universal translator for real-world data.

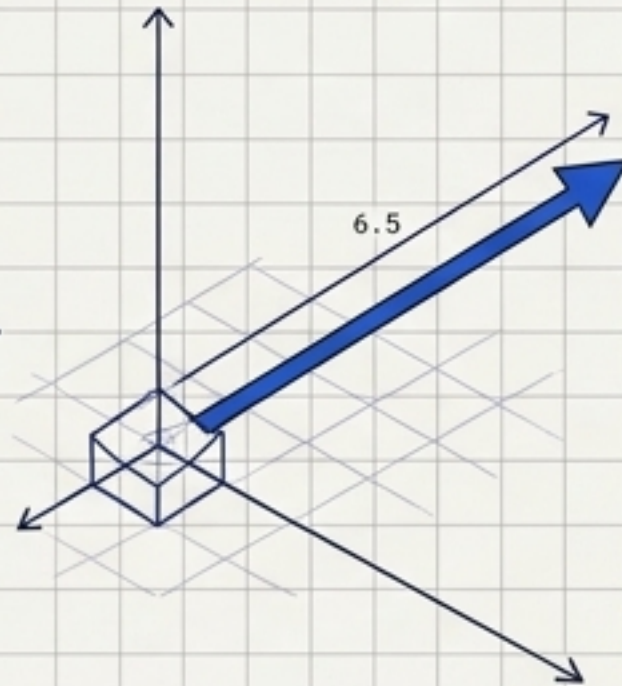
[01-linear-algebra-intuition]

0.05

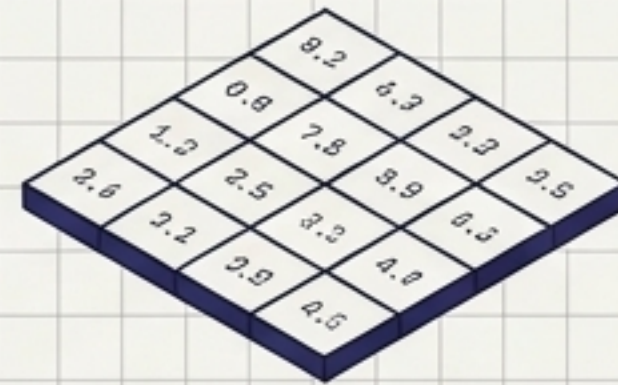


Scalar

[02-vectors-matrices-operations]

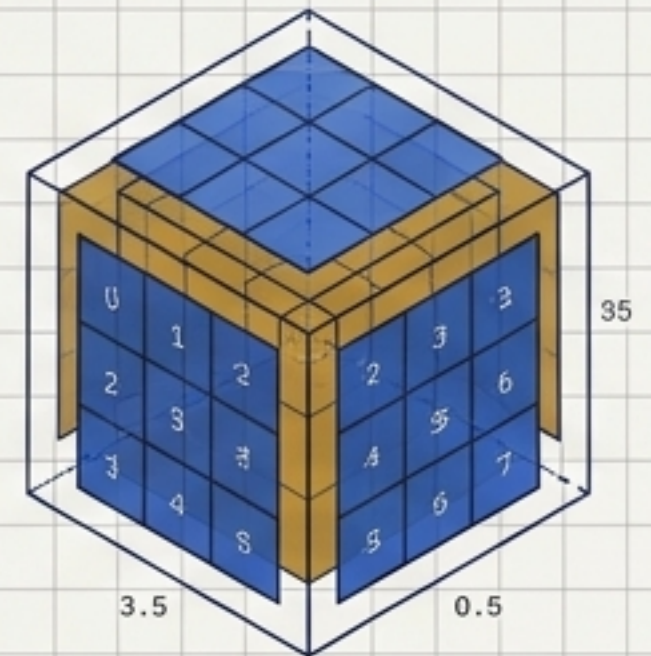


Vector



Matrix

[12-tensor-operations]

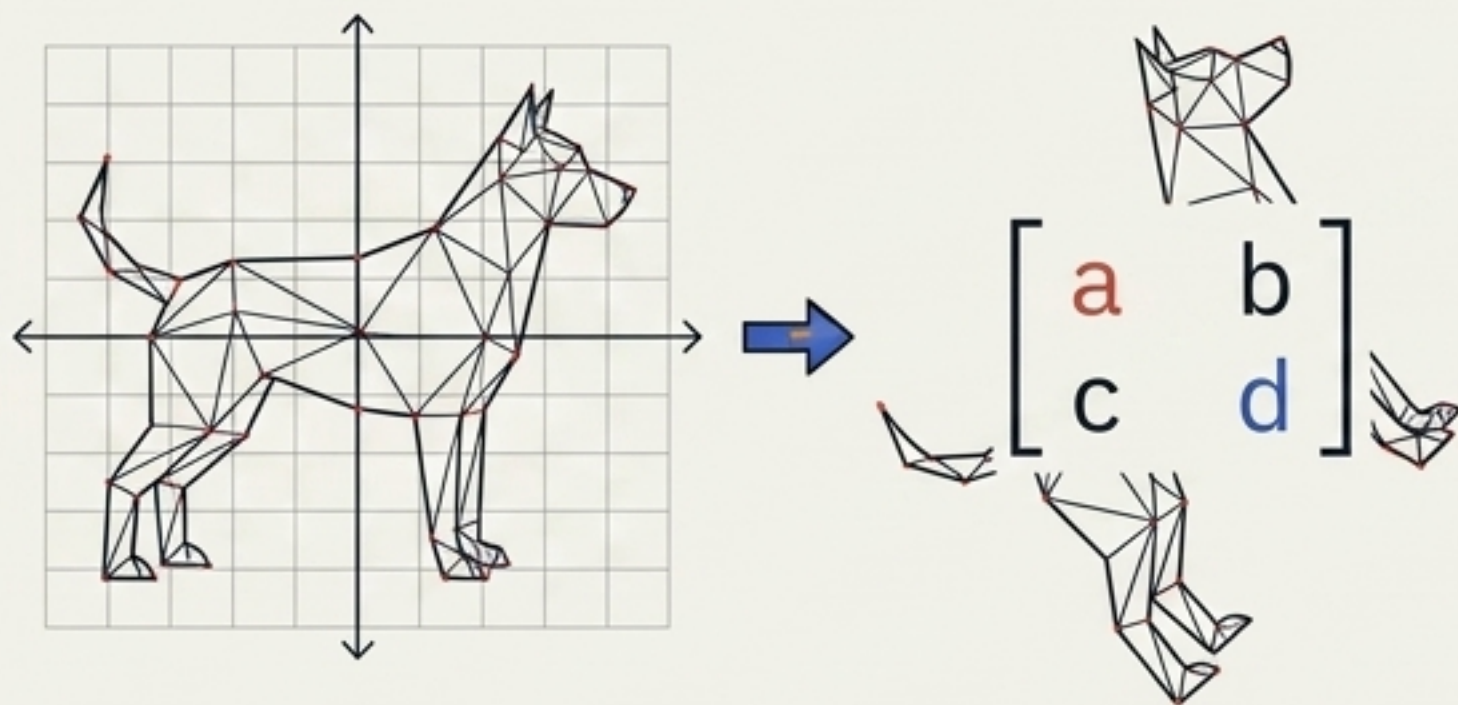


Tensor

AI possesses no innate understanding of images, audio, or text. It only processes multidimensional numbers mapped in space.

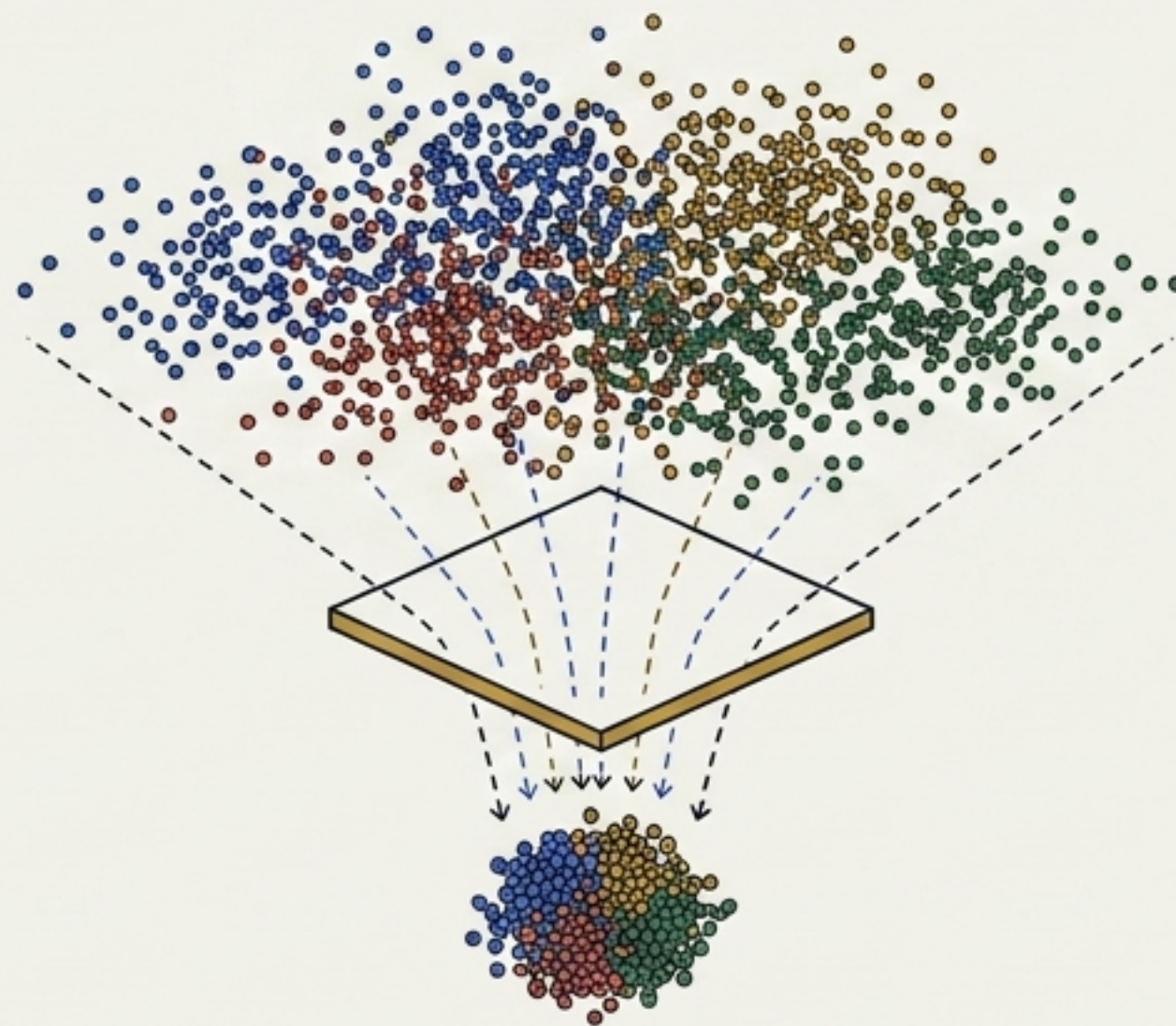
Matrix operations physically stretch, rotate, and compress informational space.

[03-matrix-transformations]



[10-dimensionality-reduction]

[11-singular-value-decomposition]

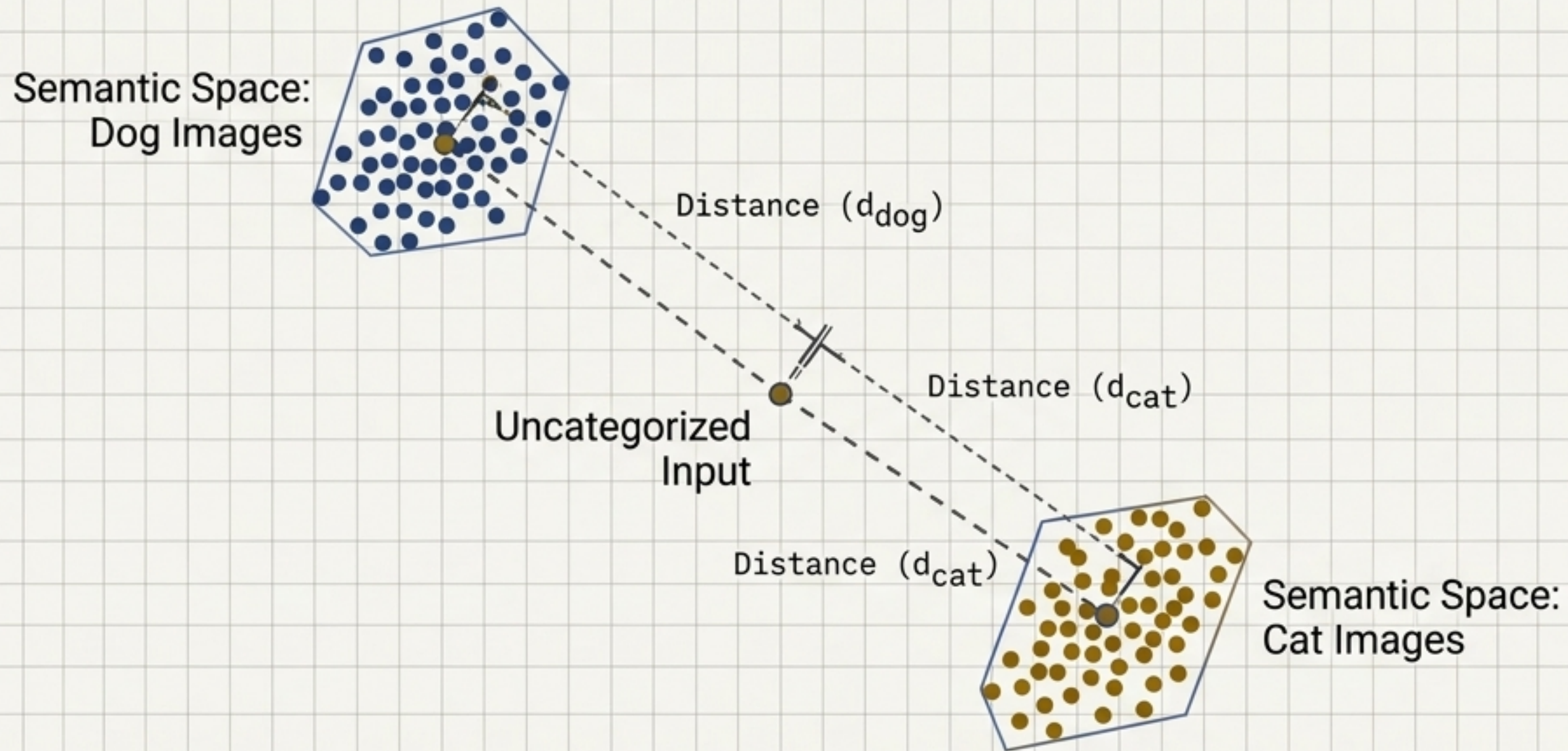


The secret behind efficient **embedding models**: finding the **mathematical essence** of data by stripping away **redundant dimensions**.

Artificial intelligence measures semantic similarity through geometric distance.

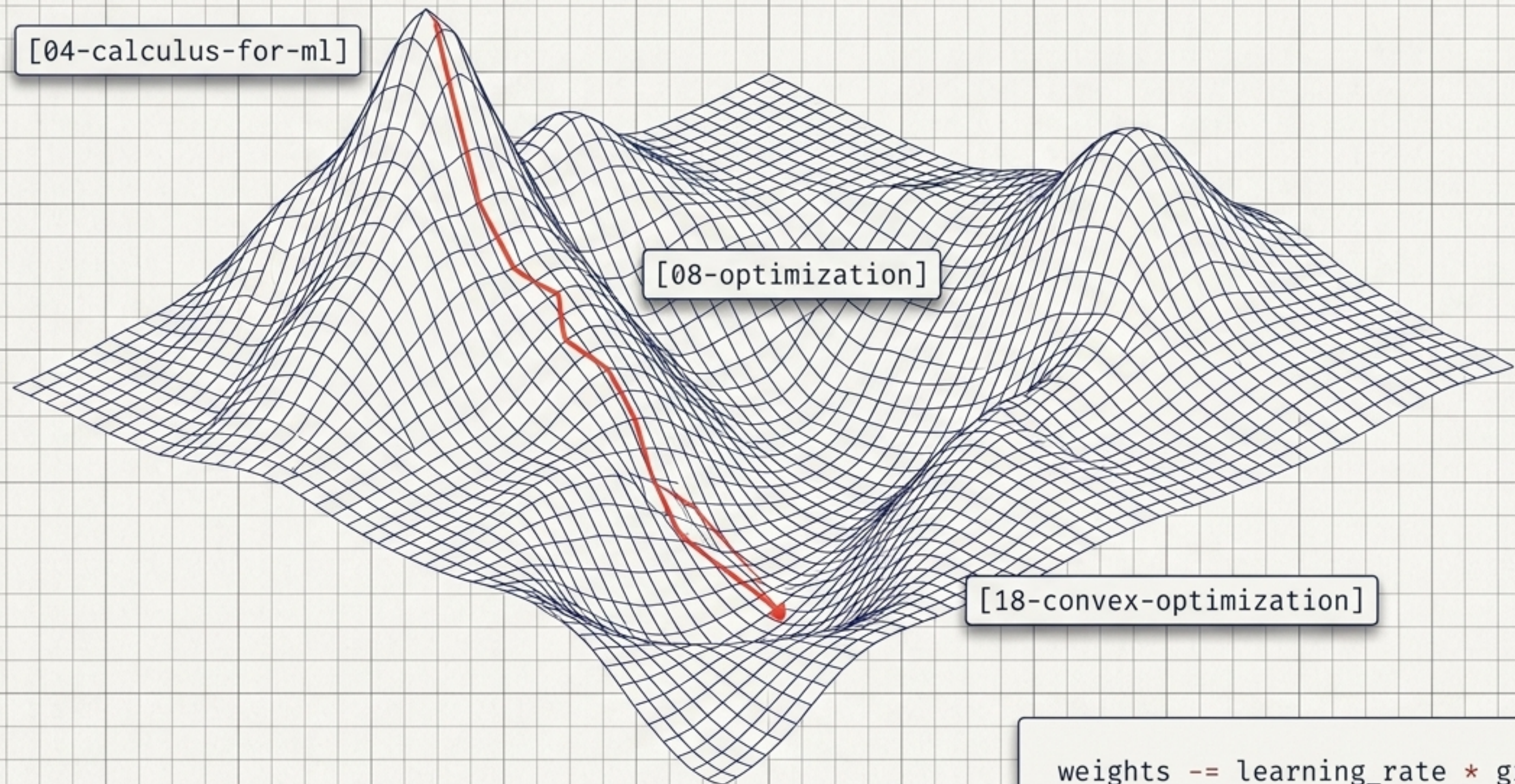
[14-norms-and-distances]

[17-linear-systems]



To an AI, 'understanding' means calculating the distance between vectors. Linear systems define the coordinates; norms define the exact distance between them.

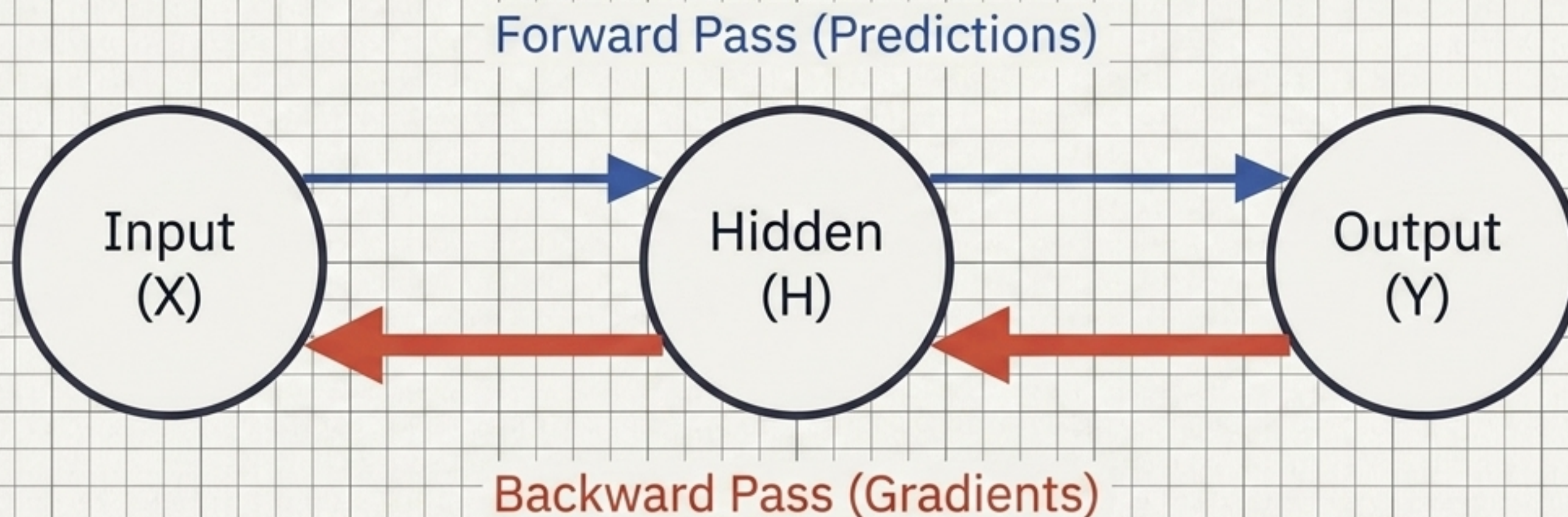
Calculus identifies the system's error, and optimization provides the path to fix it.



```
weights -= learning_rate * gradients
```

The chain rule manifests mechanically as automated backward propagation.

[05-chain-rule-and-autodiff]

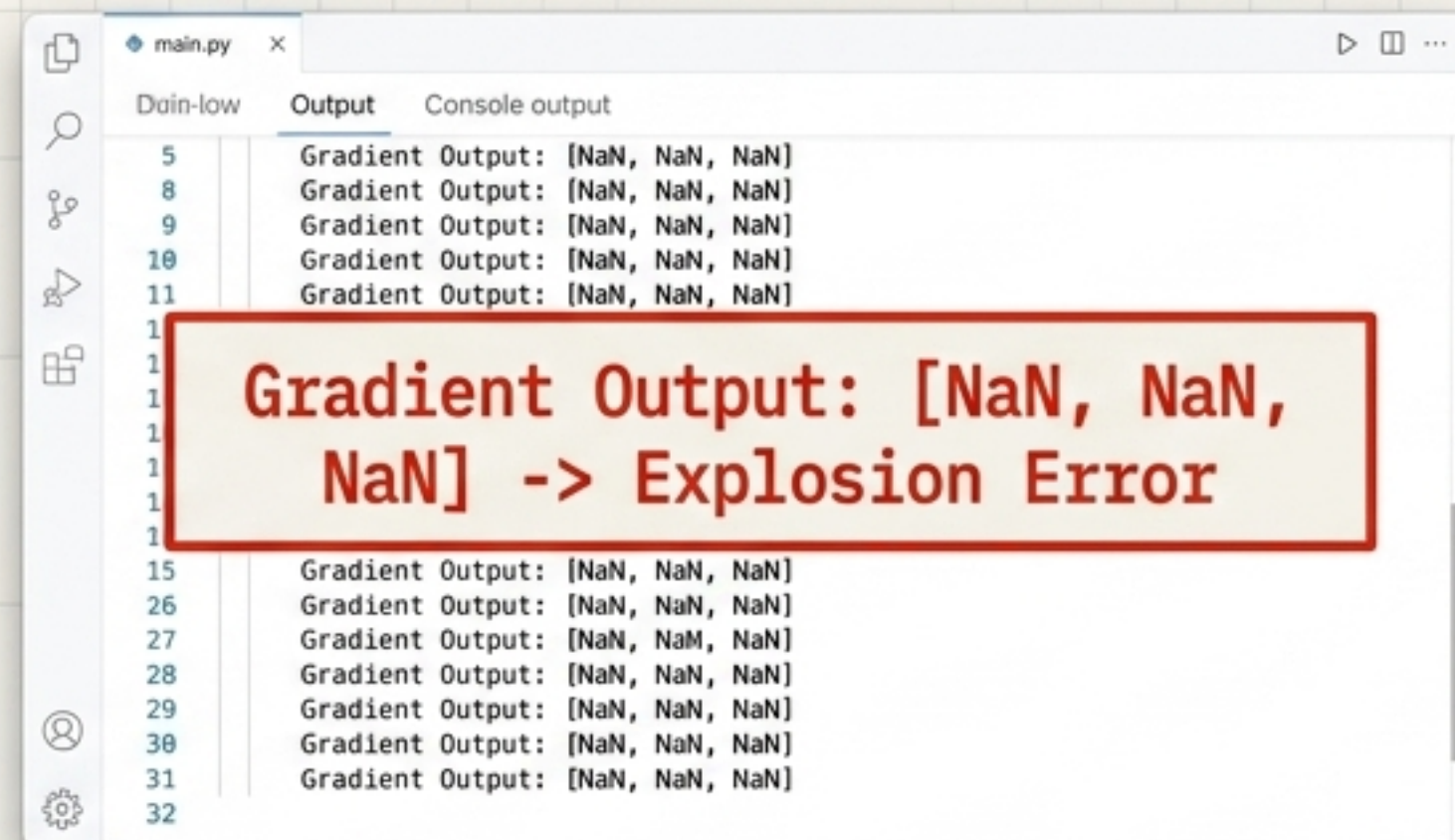


You do not need to derive complex partial derivatives by hand. You must understand how the computational graph automatically propagates errors backward to update the system.

Infinite mathematical precision shatters against finite computational memory.

$$\int_{-\infty}^{\infty} \frac{e^{-\frac{x^2}{2\sigma^2}}}{\sigma\sqrt{2\pi}} dx = 1$$

Theoretical Math: Infinite Precision

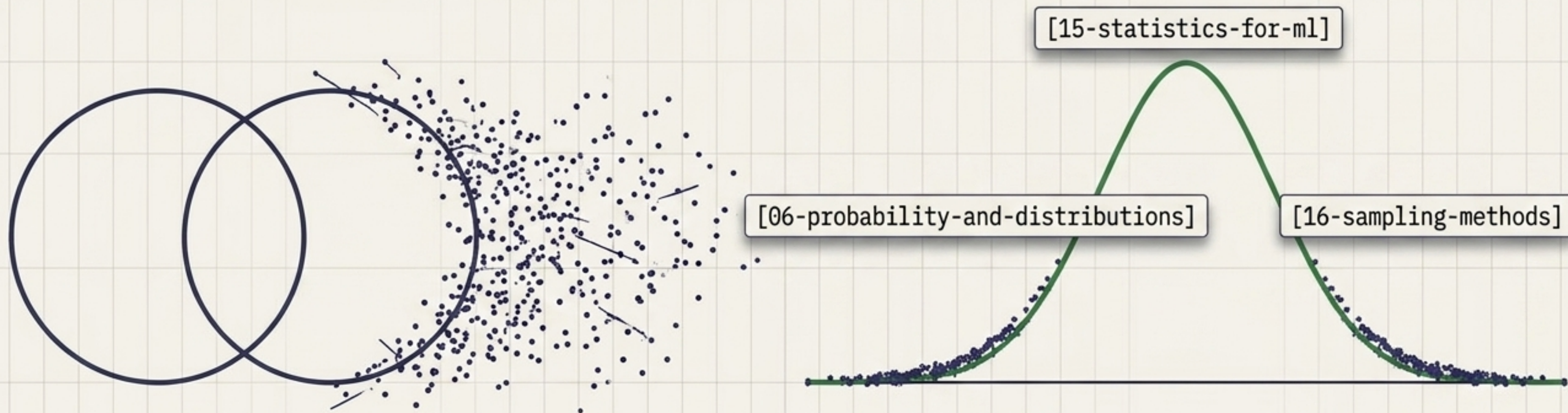


The screenshot shows a Jupyter Notebook interface with a file named 'main.py'. The 'Output' tab is active, displaying a list of 'Gradient Output' messages. The first five lines (lines 5-11) show 'Gradient Output: [NaN, NaN, NaN]'. A red-bordered box highlights a message on line 12: 'Gradient Output: [NaN, NaN, NaN] -> Explosion Error'. Below this, lines 15-32 show more 'Gradient Output' messages, some of which are '[NaN, NaN, NaN]' and others are '[NaN, NaN, NaN]'.

Engineering Reality: Memory Limits

Math assumes perfect, infinite precision. Computers use limited memory. Preventing vanishing and exploding numbers separates abstract theorists from practical engineers.

Intelligence requires mapping probabilities across an inherently uncertain world.



AI does not produce absolute certainties. It produces high-confidence probabilities.
Mastering distributions allows code to handle messy, incomplete real-world data.

Models continuously update their mathematical worldview based on new evidence.

[07-bayes-theorem]

Prior
Belief

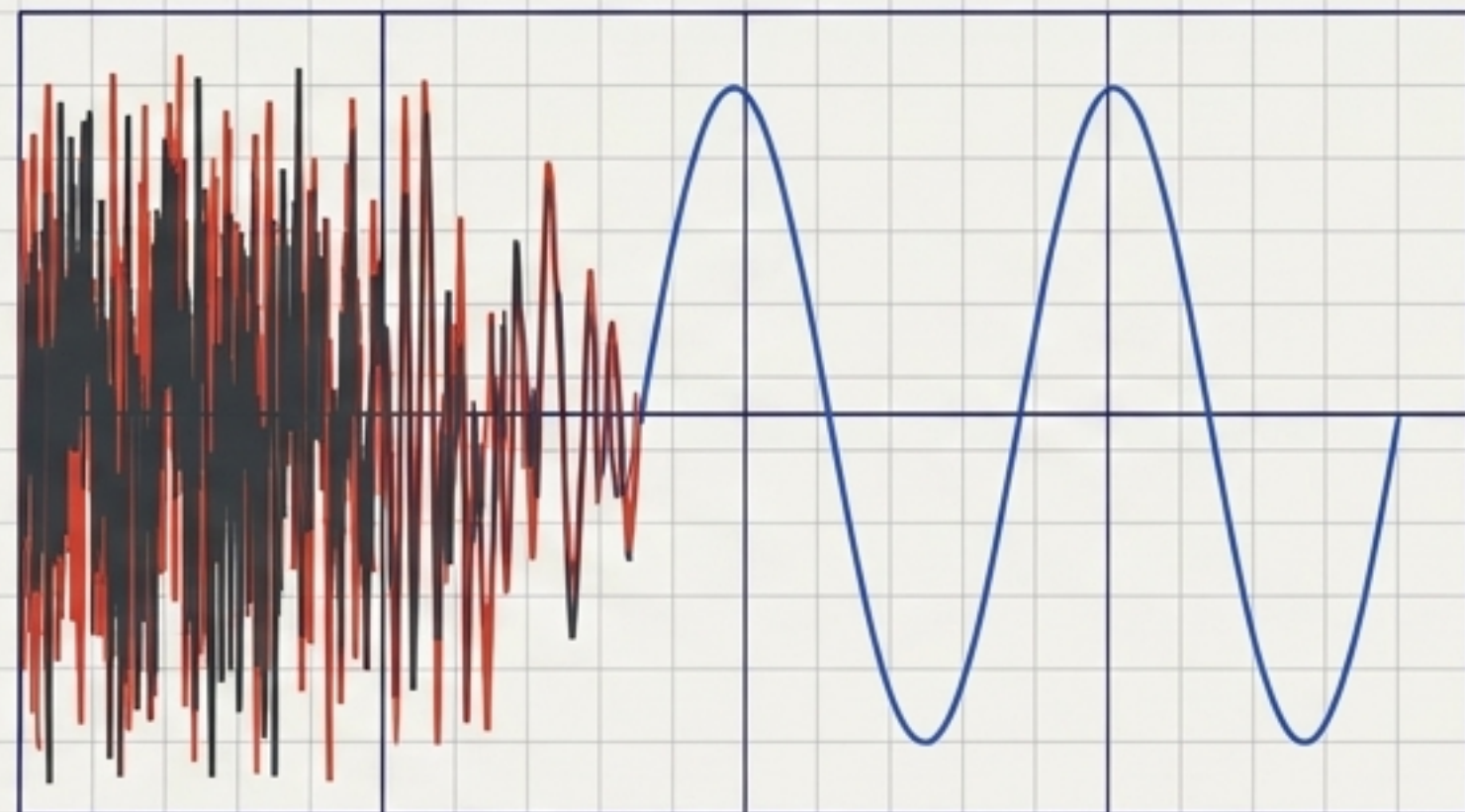
+

New
Evidence

=

Updated
Belief

09-information-theory]

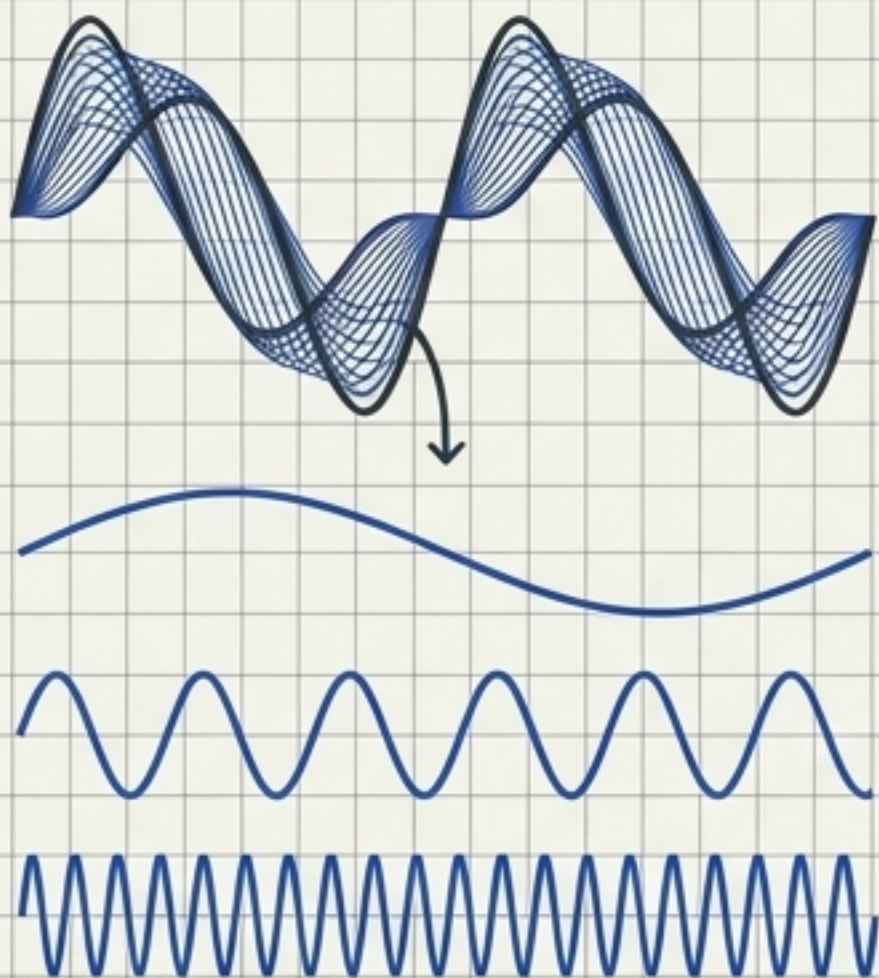


Bayes Theorem governs how algorithms adjust to reality. Information Theory measures the exact mathematical 'surprise' or value of that new reality (Entropy).

Advanced topologies decode temporal, relational, and chaotic systems.

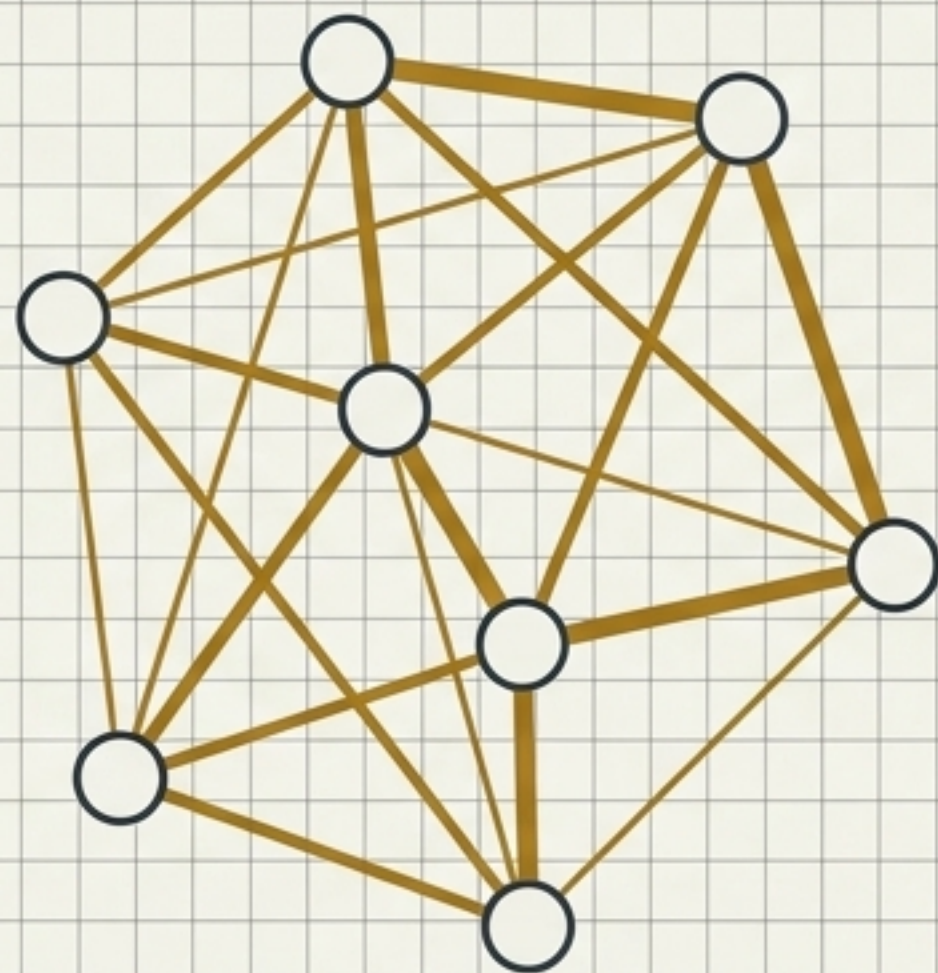
[20-fourier-transform]

[19-complex-numbers]



Time-Series Decoding

[21-graph-theory]



Relational Mapping

[22-stochastic-processes]



Random System Modeling

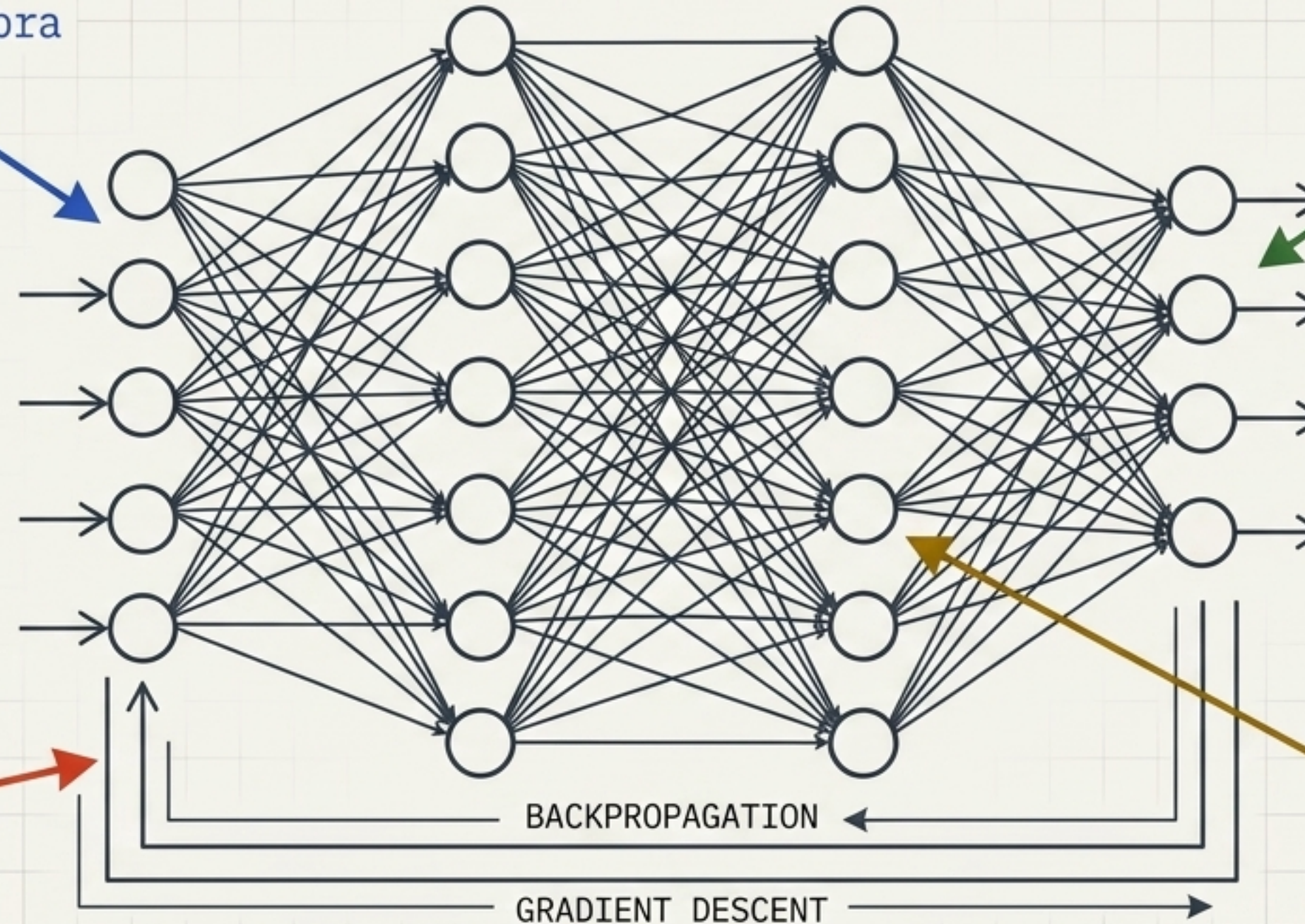
These foundational pillars combine simultaneously to forge a living neural network.

Pillar 1: Linear Algebra
translates the data.

Pillar 3: Probability
assigns the confidence
score.

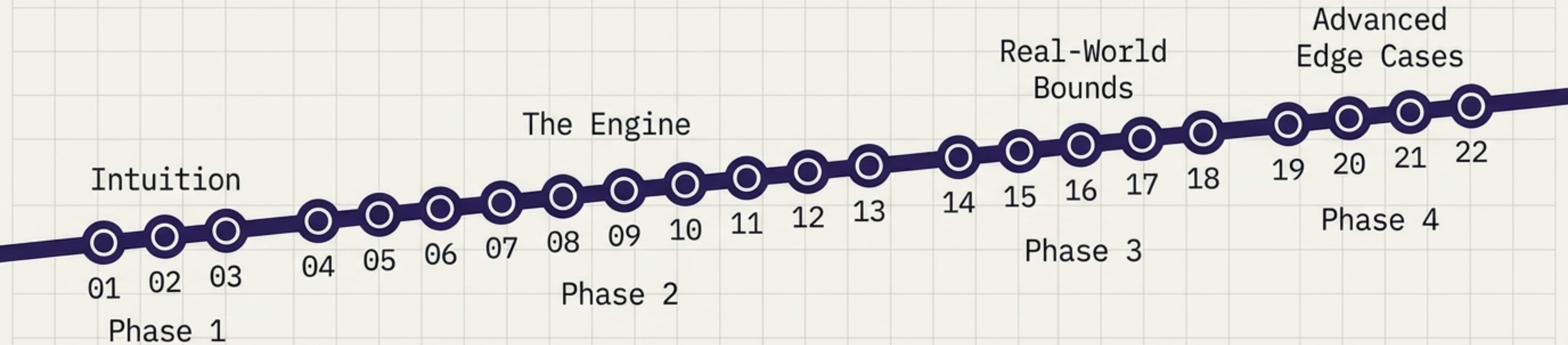
Pillar 2: Calculus
calculates the
gradient penalty.

Pillar 4: Advanced
Structures define the
topological flow.



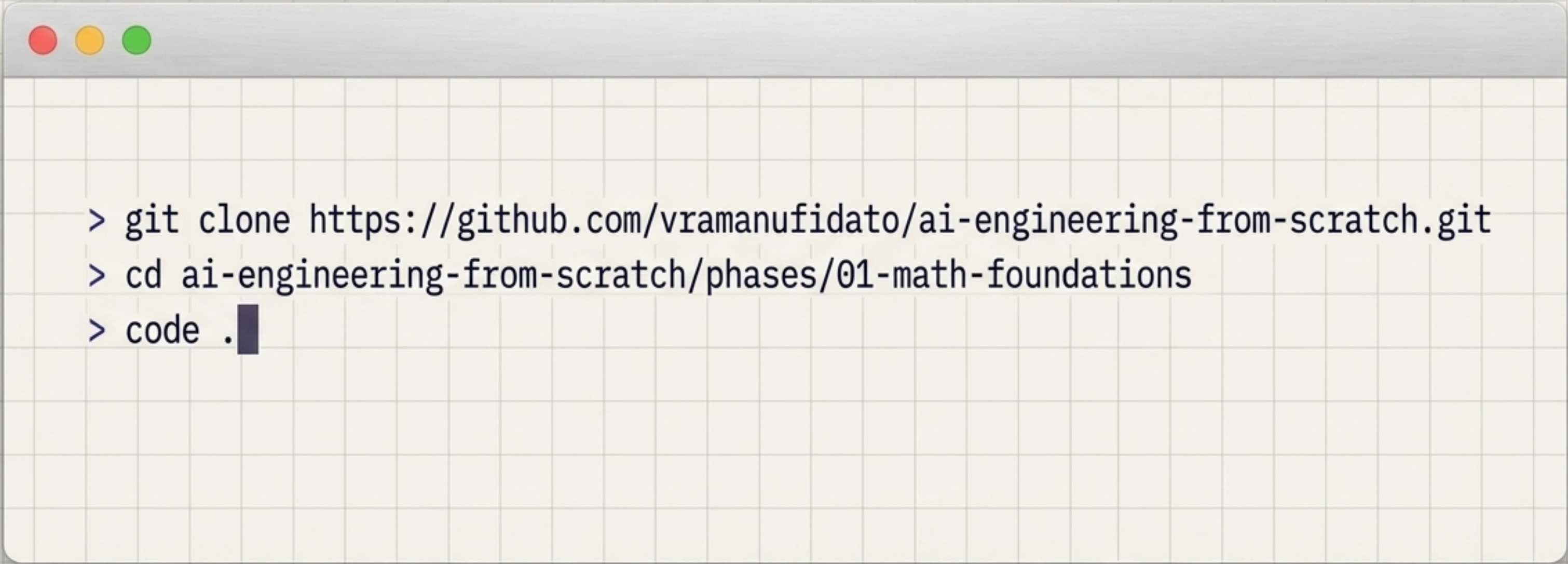
Nothing exists in isolation. A working model is the symphony of all 22 mathematical modules executing in milliseconds.

Mastering the mathematics of AI engineering requires a strict, sequential progression.



Each commit is a **mathematical concept**. Every concept is backed by **executable Python**. No skipping ahead.

Stop reading equations. Start executing them.



```
> git clone https://github.com/vramanufidato/ai-engineering-from-scratch.git  
> cd ai-engineering-from-scratch/phases/01-math-foundations  
> code .
```

Access the complete Phase 1 open-source curriculum today.